Data Structures and Algorithms

INFORMATION
TECHNOLOGY

Objectives

- In this session, you will learn to:
  - Implement a binary tree
  - Implement a binary search tree

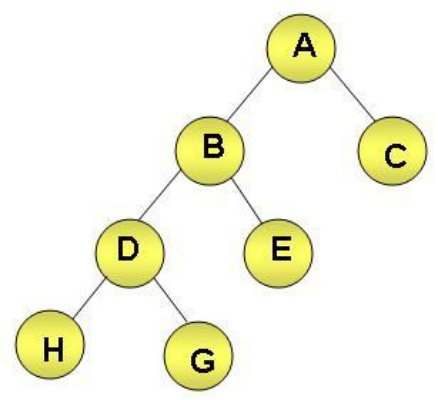
Implementing a Binary Tree

- A binary tree is a special type of tree that offers a lot of practical applications in the field of computer science.

Data Structures and Algorithms

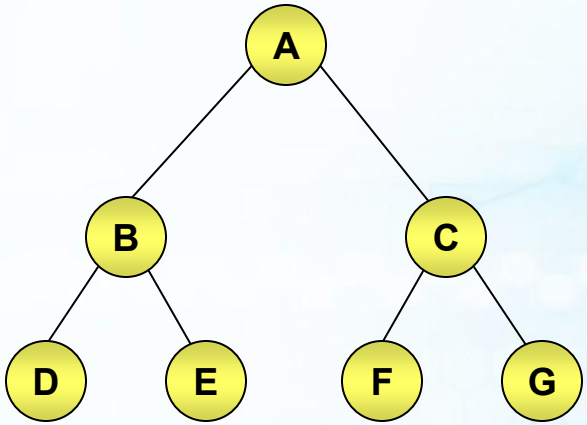
Defining Binary Trees

- Binary tree is a specific type of tree in which each node can have at most two children namely left child and right child.
- There are various types of binary trees:
  - Strictly binary tree
  - Full binary tree
  - Complete binary tree
- Strictly Binary Tree:
  - A binary tree in which every node, except for the leaf nodes, has non-empty left and right children.



Defining Binary Trees (Contd.)

- Full Binary Tree:
  - A binary tree of depth d that contains exactly $2^d - 1$ nodes.

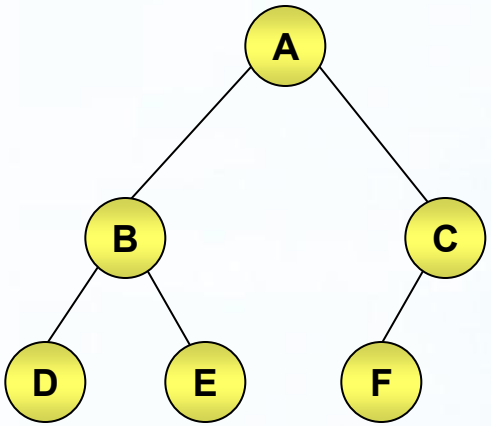


Depth = 3

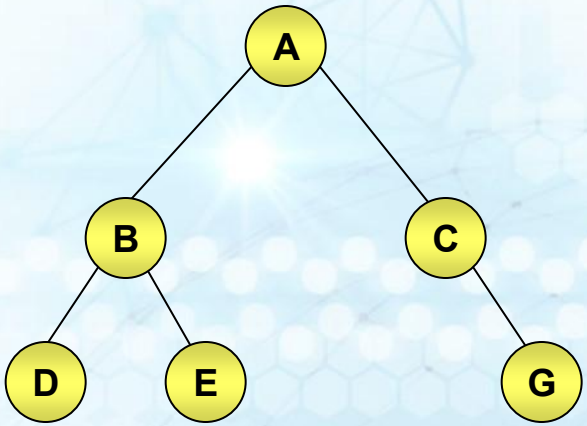
Total number of nodes = $2^3 - 1 = 7$

Defining Binary Trees (Contd.)

- Complete Binary Tree:
  - A binary tree in which all levels, except possibly the deepest level, are completely filled and at the deepest level, all nodes are as far left as possible.



Complete Binary Tree



Incomplete Binary Tree

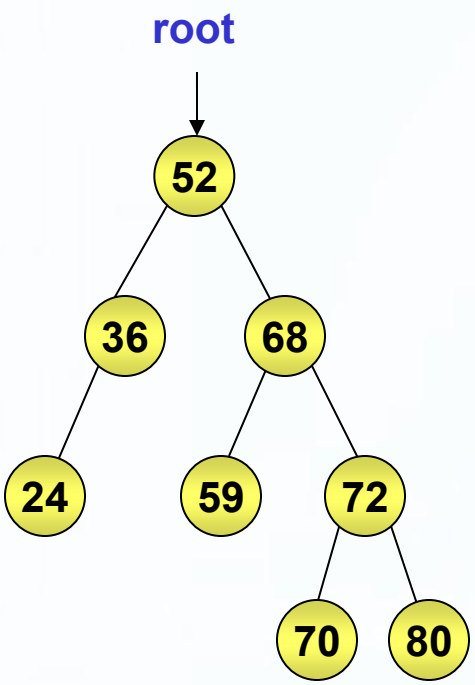
Representing a Binary Tree

- Linked Representation of a Binary Tree:
  - It uses a linked list to implement a binary tree.
  - Each node in the linked representation holds the following information:
    - Data
    - Reference to the left child
    - Reference to the right child
  - If a node does not have a left child or a right child or both, the respective left or right child fields of that node points to `NULL`.

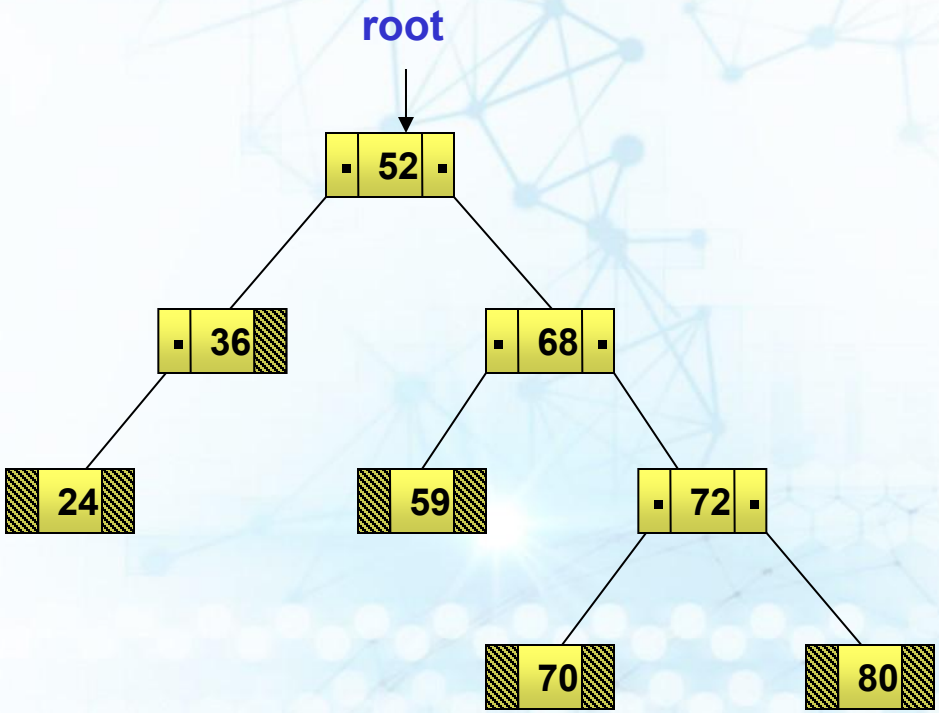
Node

Data Structures and Algorithms

Representing a Binary Tree (Contd.)



Binary Tree



Linked Representation

Representing a Binary Tree (Contd.)

- A binary tree is represented in a program by defining two classes:
  - A class to represent a node of the tree: This class represents the structure of each node in a binary tree.

```
// Code in Java:
 class Node
 {
     public int info;
     public Node lchild;
     public Node rchild;
 }
```

Data Structures and Algorithms

Representing a Binary Tree (Contd.)

```
// Code in C++:
    class Node
    {
        public int info;
        public Node lchild;
        public Node rchild;
    }
```

Representing a Binary Tree (Contd.)

- A class to represent the binary tree: This class implements various operations on a binary tree, such as insert, delete, and traverse.

```
// Code in Java:
   class BinaryTree
   {
    public Node ROOT;
    public BinaryTree()
    {
     ROOT = null;
    }
    public void insert(int element);
    public void find(int element);
```

Data Structures and Algorithms

Representing a Binary Tree (Contd.)

```
public void inorder(Node ptr);
public void preorder(Node ptr);
public void postorder(Node ptr);
};

// Code in C++:
class BinaryTree
{
  public:
  Node *ROOT;
  BinaryTree()
  {
     ROOT = NULL;
  }
```

Data Structures and Algorithms

Representing a Binary Tree (Contd.)

```
void insert(int element);
void find(int element, Node **parent,Node
  **currentNode);
void inorder(Node *ptr);
void preorder(Node *ptr);
void postorder(Node *ptr);
};
```

Traversing a Binary Tree

- You can implement various operations on a binary tree.
- A common operation on a binary tree is traversal.
- Traversal refers to the process of visiting all the nodes of a binary tree once.
- There are three ways for traversing a binary tree:
  - Inorder traversal
  - Preorder traversal
  - Postorder traversal

Traversing a Binary Tree (Contd.)

- The following steps are used for traversing a tree in inorder sequence:
  1. Traverse the left sub tree.
  2. Visit root.
  3. Traverse the right sub tree.

Let us view an animation to understand the inorder traversal of a binary tree.

Traversing a Binary Tree (Contd.)

- The following steps are used for traversing a tree in preorder sequence:
  1. Visit root.
  2. Traverse the left sub tree.
  3. Traverse the right sub tree.

Let us view an animation to understand the preorder traversal of a binary tree.

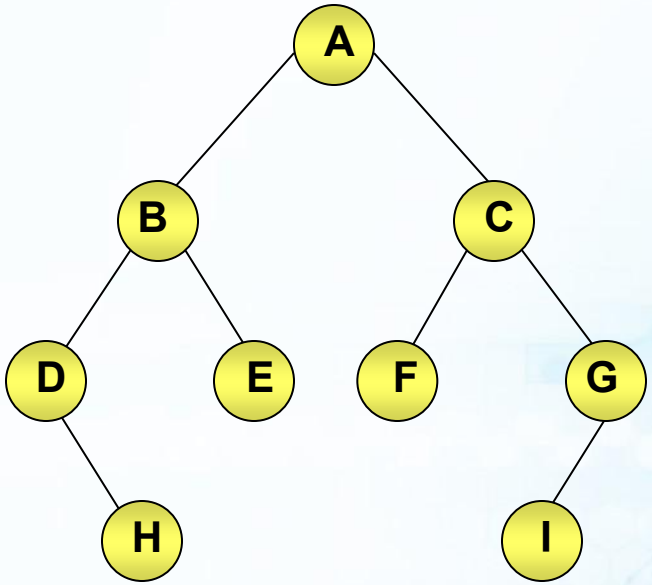
Traversing a Binary Tree (Contd.)

- The following steps are used for traversing a tree in postorder sequence:
  1. Traverse the left sub tree.
  2. Traverse the right sub tree.
  3. Visit the root.

Data Structures and Algorithms

Traversing a Binary Tree (Contd.)

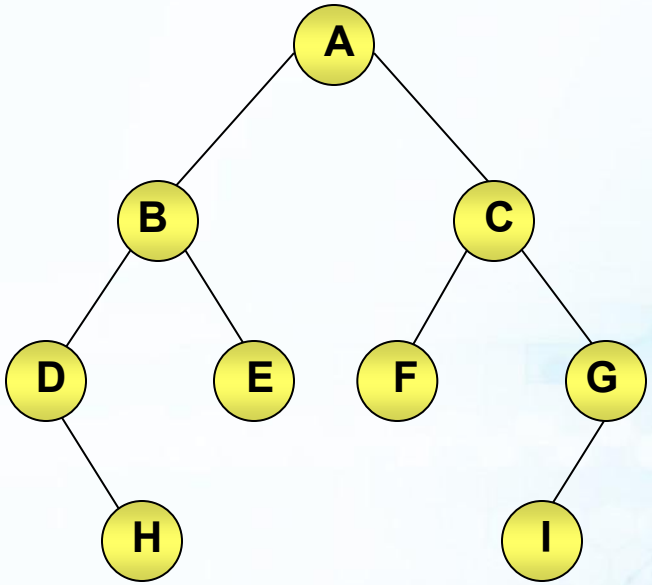
- Perform the postorder traversal of the following tree.



Data Structures and Algorithms

Traversing a Binary Tree (Contd.)

- Perform the postorder traversal of the following tree.



Postorder Traversal: H D E B F I G C A

Just a Minute

- In the _____ traversal method, the root is processed before traversing the left and right sub trees.

Just a Minute (Contd.)

- Solution:
  - Preorder

Implementing a Binary Search Tree

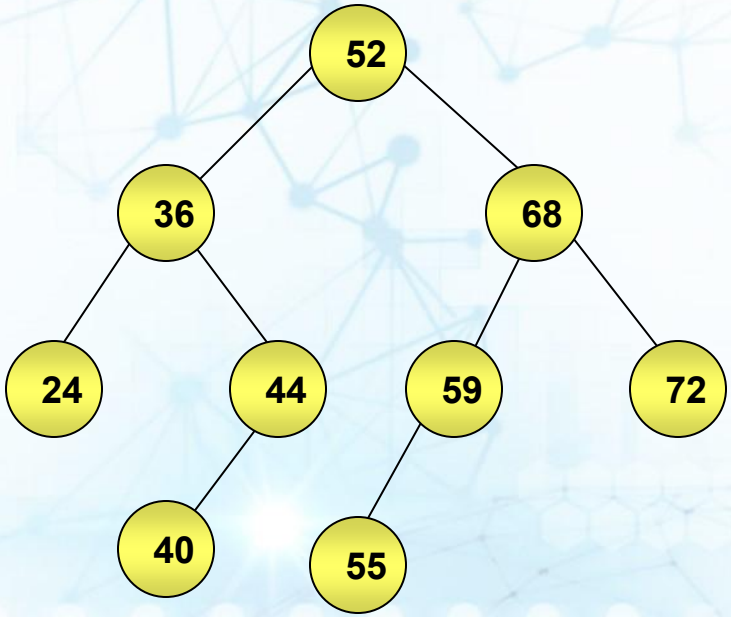
- A common variant of binary tree is a binary search tree.
- A binary search tree matches the search speed of arrays.
- It offers efficient insertions and deletions as in the case of a linked list.

Defining a Binary Search Tree

- Binary search tree is a binary tree in which every node satisfies the following conditions:
  - All values in the left subtree of a node are less than the value of the node.
  - All values in the right subtree of a node are greater than the value of the node.

Defining a Binary Search Tree (Contd.)

- The following is an example of a binary search tree.



- You can implement various operations on a binary search tree:
  - Traversal
  - Search
  - Insert
  - Delete

Searching a Node in a Binary Search Tree

- Search operation refers to the process of searching for a specified value in the tree. To search for a specific value, you need to perform the following steps:

```
1. Make currentNode point to the root node.
2. If currentNode is null:
    a. Display "Not Found".
    b. Exit.
3. Compare the value to be searched with the value of
   currentNode. Depending on the result of the
   comparison, there can be three possibilities:
    a. If the value is equal to the value of currentNode:
        i. Display "Found".
        ii. Exit.
```

Searching a Node in a Binary Search Tree (Contd.)

b. If the value is less than the value of currentNode:
   i. Make currentNode point to its left child.
   ii. Go to step 2.

c. If the value is greater than the value of currentNode:
   i. Make currentNode point to its right child.
   ii. Go to step 2.

Just a Minute

- In a binary search tree, all values in the left sub tree of a node are _____ than or equal to the value of the node.

Just a Minute (Contd.)

- Solution:
  - Smaller

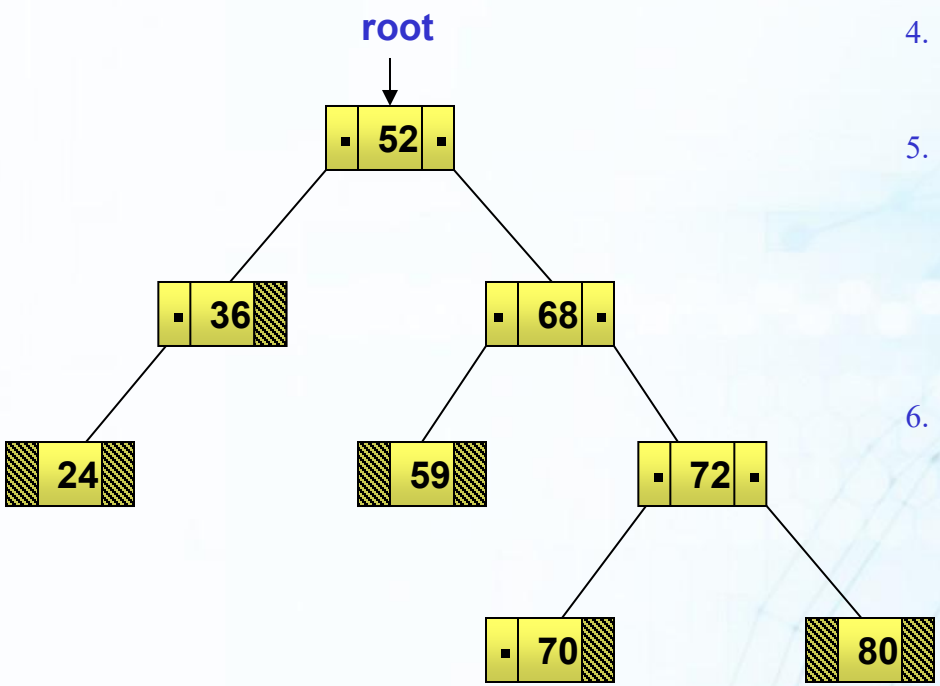
Inserting Nodes in a Binary Search Tree

- Before inserting a node in a binary search tree, you first need to check whether the tree is empty or not.
- If the tree is empty, make the new node as root.
- If the tree is not empty, you need to locate the appropriate position for the new node to be inserted.
- This requires you to locate the parent of the new node to be inserted.
- Once the parent is located, the new node is inserted as the left child or right child of the parent.
- To locate the parent of the new node to be inserted, you need to implement a search operation on the tree.

Data Structures and Algorithms

Inserting Nodes in a Binary Search Tree (Contd.)

- Algorithm to locate the parent of the new node to be inserted:



1. Mark the root node as currentNode.
2. Make parent point to NULL.
3. Repeat steps 4, 5, and 6 until currentNode becomes NULL.
4. Make parent point to currentNode.
5. If the value of the new node is less than or equal to that of currentNode:

   Make currentNode point to its left child.

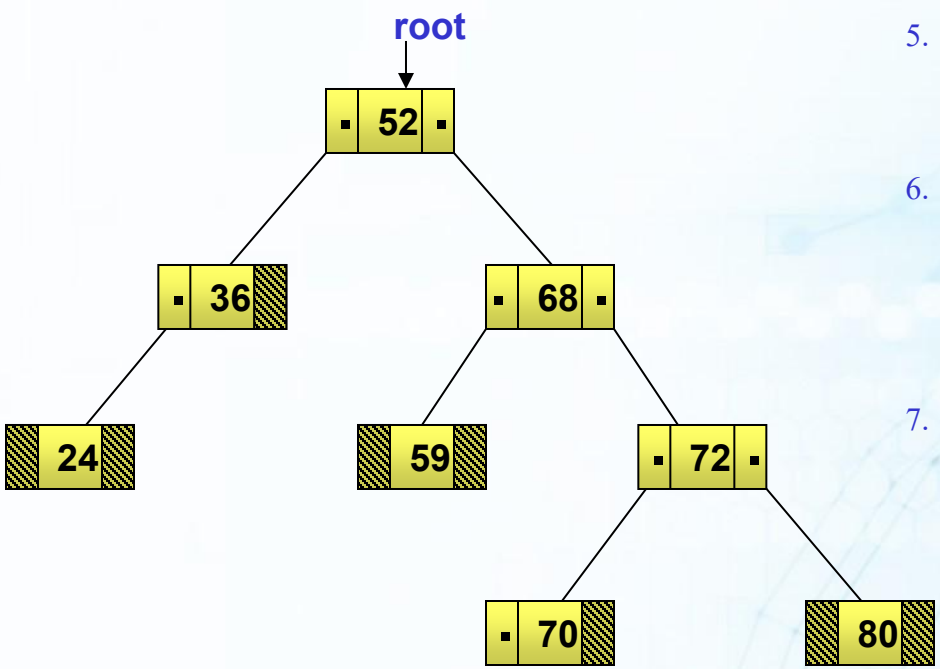
6. If the value of the new node is greater than that of currentNode:

   Make currentNode point to its right child.

Data Structures and Algorithms

Inserting Nodes in a Binary Search Tree (Contd.)

- Algorithm to insert a node in a binary search tree:



1. Allocate memory for the new node.
2. Assign value to the data field of new node.
3. Make the left and right child of the new node point to NULL.
4. Locate the node which will be the parent of the node to be inserted. Mark it as parent.
5. If parent is NULL (Tree is empty):
   a. Mark the new node as ROOT.
   b. Exit.
6. If the value in the data field of new node is less than that of parent:
   a. Make the left child of parent point to the new node.
   b. Exit.
7. If the value in the data field of the new node is greater than that of the parent:
   a. Make the right child of parent point to the new node.
   b. Exit.

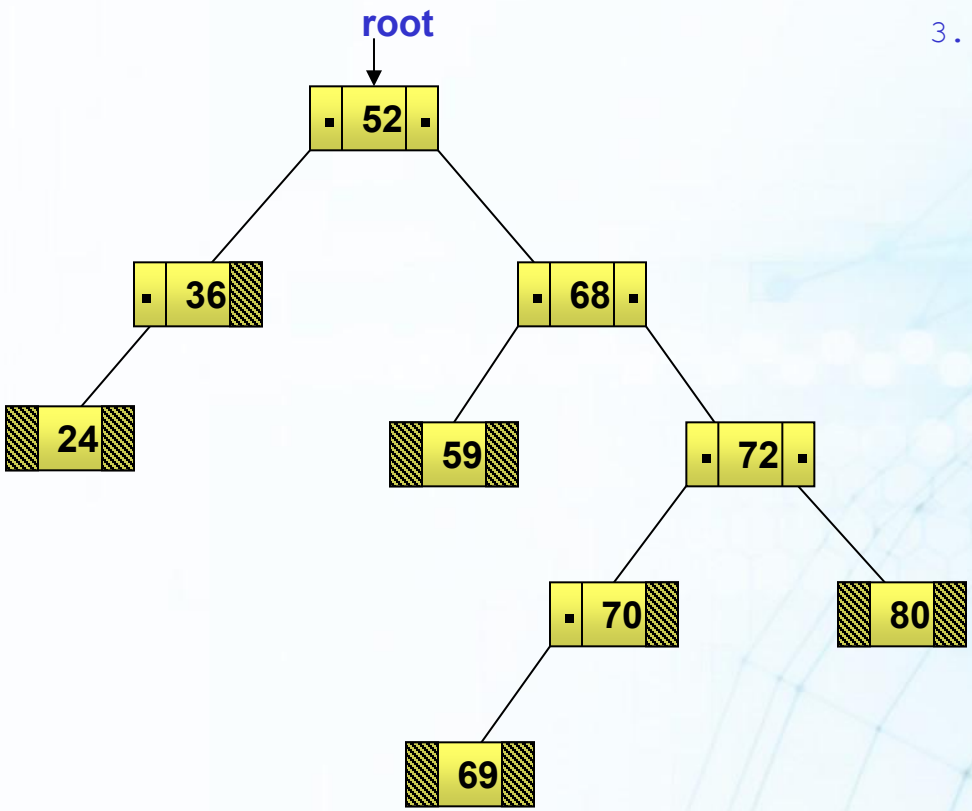
Deleting Nodes from a Binary Search Tree

- Delete operation in a binary search tree refers to the process of deleting the specified node from the tree.
- Before implementing a delete operation, you first need to locate the position of the node to be deleted and its parent.
- To locate the position of the node to be deleted and its parent, you need to implement a search operation.

Data Structures and Algorithms

Deleting Nodes from a Binary Search Tree (Contd.)

- Algorithm to locate the node to be deleted:



1. Make a variable/pointer currentNode point to the ROOT node.

2. Make a variable/pointer parent point to NULL.

3. Repeat steps a, b, and c until currentNode becomes NULL or the value of the node to be searched becomes equal to that of currentNode:

   a. Make parent point to currentNode.

   b. If the value to be deleted is less than that of currentNode:

      Make currentNode point to its left child.

   c. If the value to be deleted is greater than that of currentNode:

      Make currentNode point to its right child.

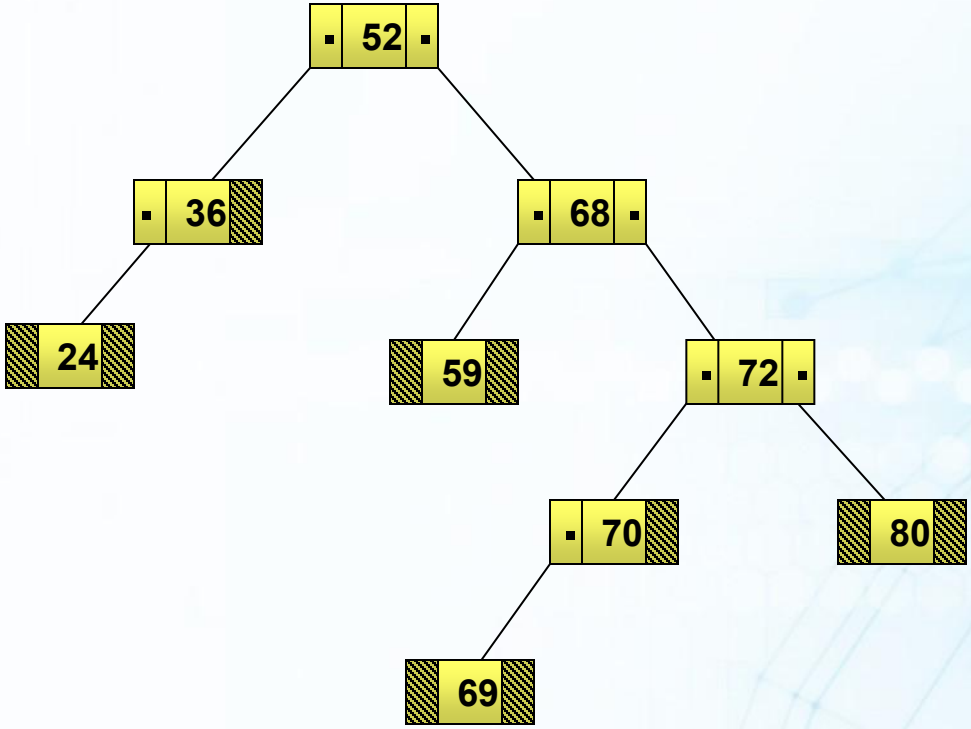
Deleting Nodes from a Binary Search Tree (Contd.)

- Once the nodes are located, there can be three cases:
  - Case I: Node to be deleted is the leaf node
  - Case II: Node to be deleted has one child (left or right)
  - Case III: Node to be deleted has two children

Data Structures and Algorithms

Deleting Nodes from a Binary Search Tree (Contd.)

- Algorithm to delete a leaf node from the binary tree:

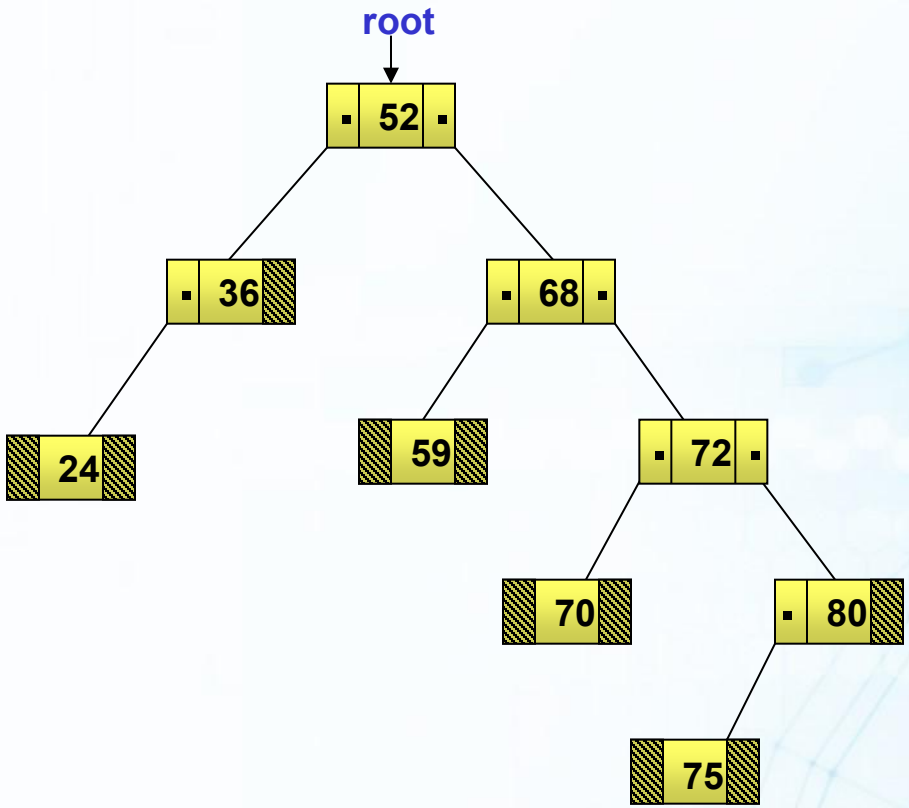


1. Locate the node to be deleted. Mark it as currentNode and its parent as parent.
2. If currentNode is the root node: // If parent is NULL
   a. Make ROOT point to NULL.
   b. Go to step 5.
3. If currentNode is the left child of parent:
   a. Make the left child field of parent point to NULL.
   b. Go to step 5.
4. If currentNode is the right child of parent:
   a. Make the right child field of parent point to NULL.
   b. Go to step 5.
5. Release the memory for currentNode.

Data Structures and Algorithms

Deleting Nodes from a Binary Search Tree (Contd.)

- Algorithm to delete a node with one child:

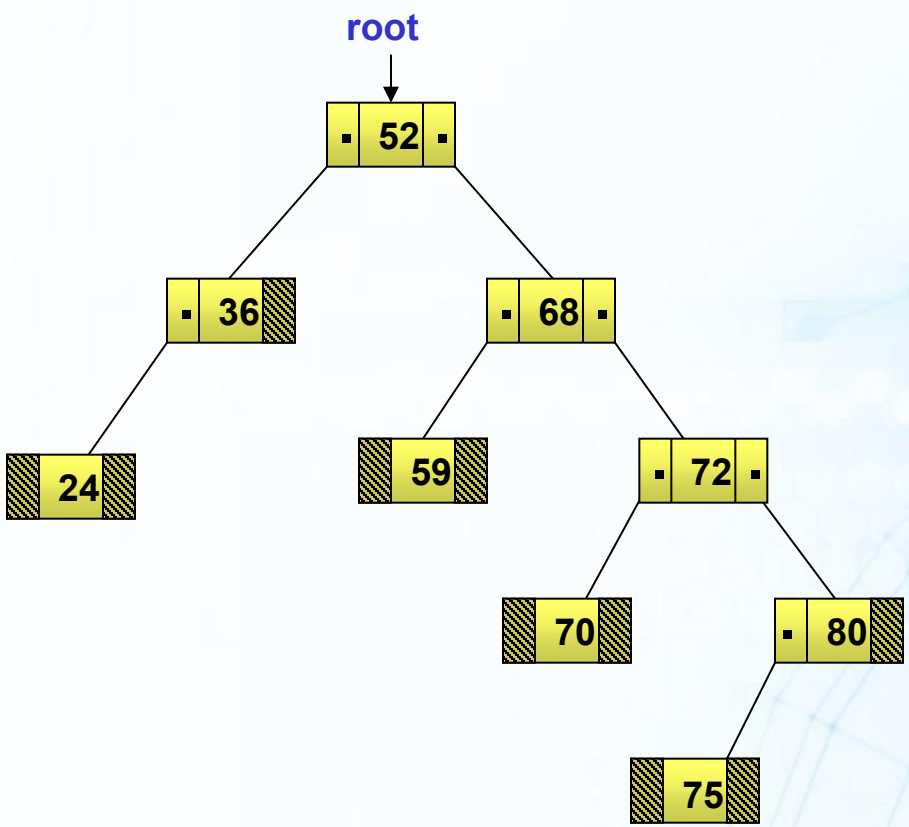


1. Locate the node to be deleted. Mark it as currentNode and its parent as parent.
2. If currentNode has a left child:
   a. Mark the left child of currentNode as child.
   b. Go to step 4.
3. If currentNode has a right child:
   a. Mark the right child of currentNode as child.
   b. Go to step 4.
4. If currentNode is the root node:
   a. Mark child as root.
   b. Go to step 7.
5. If currentNode is the left child of parent:
   a. Make left child field of parent point to child.
   b. Go to step 7.
6. If currentNode is the right child of parent:
   a. Make right child field of parent point to child.
   b. Go to step 7.
7. Release the memory of currentNode.

Data Structures and Algorithms

Deleting Nodes from a Binary Search Tree (Contd.)

- Algorithm to delete a node with two children:



1. Locate the node to be deleted. Mark it as currentNode and its parent as parent.
2. Locate the inorder successor of currentNode. Mark it as Inorder_suc. Execute the following steps to locate Inorder_suc:
   a. Mark the right child of currentNode as Inorder_suc.
   b. Repeat until the left child of Inorder_suc becomes NULL:
      Make Inorder_suc point to its left child.
3. Replace the information held by currentNode with that of Inorder_suc.
4. If the node marked Inorder_suc is a leaf node:

   Delete the node marked Inorder_suc by using the algorithm for Case I.

5. If the node marked Inorder_suc has one child:

   Delete the node marked Inorder_suc by using the algorithm for Case II.

Demo: Implementing a Binary Search Tree

- Problem Statement:
  - Write a program to implement the insert and traverse operations on a binary search tree that contains the words in a dictionary.

Demo: Implementing a Binary Search Tree (Contd.)

- Solution:
  - To implement insert and traverse operations on a binary search tree, you need to perform the following tasks:
    1. Write the code.
    2. Execute the code and verify the output.

Summary

- In this session, you learned that:
  - A binary tree is a specific type of tree in which each node can have a maximum of two children.
  - Binary trees can be implemented by using arrays, as well as linked lists, depending upon the requirement.
  - Traversal of a tree is the process of visiting all the nodes of the tree once.
  - There are three types of traversals: inorder traversal, preorder traversal, and postorder traversal.
  - Binary search tree is a binary tree in which the value of the left child of a node is always less than or equal to the value of the node, and the value of the right child of a node is greater than the value of the node.